UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2/20252026

SECP3843 - SPECIAL TOPIC IN

DATA ENGINEERING

SECTION 01

TUTORIAL 3: AI ALGORITHM

GROUP MEMBERS:

NAME	MATRIC ID
MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205

LECTURER:

Dr. Aryati Binti Bakri

Table of Contents

1.0 Image Classification

Image Processing (IP) stands as a multifaceted field encompassing a range of methodologies dedicated to gleaning valuable insights from images. Concurrently, the landscape of Artificial Intelligence (AI) has burgeoned into an expansive realm of exploration, serving as the conduit through which intelligent machines strive to replicate human cognitive capacities (Archana & Jeevaraj, 2024). Another study from GeeksforGeeks (2025) states that image classification is the process of assigning a predefined label to an image based on its visual content.

One of the main challenges in image classification is enabling AI models to interpret visual information and distinguish between different image categories. Unlike humans, who naturally process visual information through their visual and cognitive systems, AI models must learn these distinctions from training data. The objective is to enable a model to automatically identify patterns, textures, shapes, and other visual features in order to classify images accurately into predefined categories.

Traditional rule-based image classification relies on manually defined rules and feature thresholds developed by domain experts. In contrast, statistical and machine learning-based classification approaches learn patterns directly from labeled training data. Although these approaches still require annotated datasets for training, they significantly reduce the need for manually designing classification rules and can achieve higher classification accuracy when applied to large and complex image datasets.

Sources:

1. Archana, R., & Jeevaraj, P. E. (2024). Deep learning models for digital image processing: a review. *Artificial intelligence review*, *57*(1), 11.
2. GeeksforGeeks. (2025, November 3). What is Image Classification? GeeksforGeeks. https://www.geeksforgeeks.org/computer-vision/what-is-image-classification/

2.0 CNN

In the field of Deep Learning (DL), the Convolutional Neural Network (CNN) is the most famous and commonly employed algorithm (Alzubaidi et al., 2021). The main benefit of CNN compared to its predecessors is that it automatically identifies the relevant features without any human supervision. CNNs have been extensively applied in a range of different fields, including computer vision, speech processing, Face Recognition, etc. The structure of CNNs was inspired by neurons in human and animal brains, similar to a conventional neural network. More specifically, in a cat's brain, a complex sequence of cells forms the visual cortex; this sequence is simulated by the CNN. A CNN consists of multiple layers, such as convolutional, pooling, and fully connected layers, which work together to extract and learn hierarchical features from input data. By progressively capturing simple features such as edges and textures before identifying more complex patterns and objects, CNNs achieve high accuracy in image classification and recognition tasks.

Source:

1. Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., ... & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. Journal of big Data, 8(1), 53.

3.0 Evaluation of CNN

Evaluating the performance of a Convolutional Neural Network (CNN) is a critical step in any machine learning workflow. After training a model on the CIFAR-10 dataset, several standard evaluation metrics are applied to measure how well the model generalises to unseen data. The three primary evaluation tools used in this tutorial are the Classification Report, the Confusion Matrix Heatmap, and the Overall Accuracy and Loss score.

4.0 Steps Hand-on in Python

Step 1: Import Libraries

The first step is to import all necessary Python libraries. TensorFlow and Keras are used to build and train neural networks. NumPy handles numerical array operations, and Matplotlib is used for visualising sample images. Figure 4.1 show the imported libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 4.1: Imported Libraries

Step 2: Load and Explore the CIFAR-10 Dataset

The CIFAR-10 dataset is loaded directly from the Keras datasets module. It is automatically split into 50,000 training images and 10,000 test images, each of size 32×32 pixels with 3 colour channels (RGB). The dataset covers 10 object classes. The labels are initially stored as 2D arrays of shape (N, 1). They are reshaped into 1D arrays to be compatible with the loss function used during training. Figure 4.2 shows the syntax of load CIFAR-10 Datasets.

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

```
x_test.shape
```

```
(10000, 32, 32, 3)
```

```
x_train.shape
```

```
(50000, 32, 32, 3)
```

```
y_train.shape
```

```
(50000, 1)
```

```
y_train[:5]
```

```
array([[6],
       [9],
       [9],
       [4],
       [1]], dtype=uint8)
```

```
y_train =  y_train.reshape(-1,)
y_train[:5]
```

```
array([6, 9, 9, 4, 1], dtype=uint8)
```

```
y_test = y_test.reshape(-1,)
```

Figure 4.2: Load CIFAR-10 Datasets

Step 3: Visualise Sample Images

A helper function is defined to display individual images from the dataset alongside their correct class label. This step helps verify that the data has been loaded correctly and provides an intuitive understanding of what the model must learn to classify. Figure 4.3 shows the helper function.

```
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

```
def plot_sample(x, y, index):
    plt.figure(figsize=(15,2))
    plt.imshow(x[index])
    plt.xlabel(classes[y[index]])
```

Figure 4.3: Helper Function

Figure 4.4-4.7 shows the sample image from the visualization function.

```
plot_sample(x_train, y_train, 5)
```

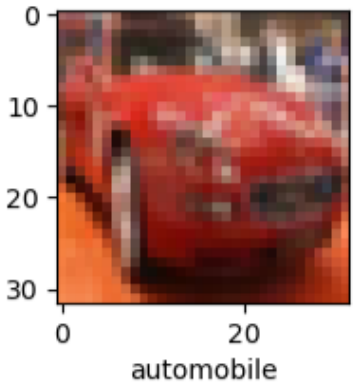

Figure 4.4: Automobile image

```
plot_sample(x_train, y_train, 10)
```

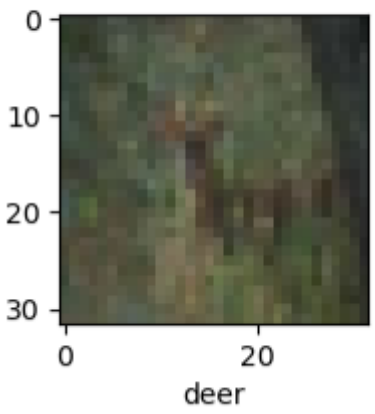

Figure 4.5: Deer image

```
plot_sample(x_train, y_train, 11)
```

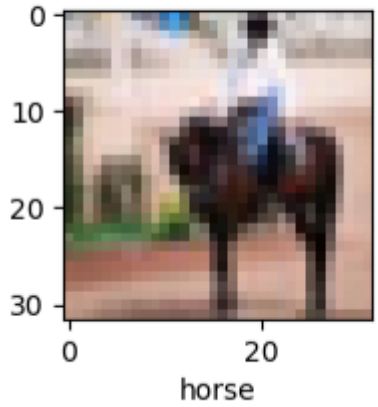

Figure 4.4: Automobile image

```
plot_sample(x_train, y_train, 501)
```

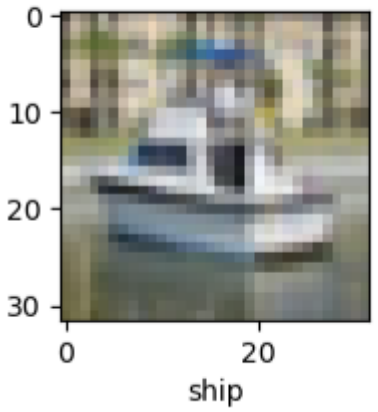

Figure 4.5: Deer image

5.0 Weakness on Current CNN Model

5.1 Immediate Spatial Flattening Destroys Image Structure

The most critical weakness of the baseline model is the Flatten layer placed at the very beginning of the network. This operation converts the 32×32×3 image tensor into a single one-dimensional vector. In doing so, all spatial relationships between neighbouring pixels are permanently destroyed before any learning takes place. Two pixels that are adjacent in the original image become completely independent inputs to the Dense layers that follow. The network therefore has no mechanism to detect edges, corners, textures, or any other spatial pattern. It can only attempt to memorise which pixel positions tend to have certain colour values for each class, a strategy that generalises very poorly.

5.2 No Feature Extraction Layers

Dense layers treat every input as an independent feature of equal importance. Unlike convolutional layers, they do not apply shared, sliding filters that can detect the same visual pattern regardless of where it appears in the image. Without any convolutional or pooling layers, the baseline model lacks the ability to build a hierarchical representation of features: from simple edges in early layers to complex object parts in deeper layers. This is the defining architectural advantage of a true CNN, and the baseline model completely lacks it.

5.3 Lack of Spatial Invariance

Because the baseline model memorises raw pixel coordinates rather than learning abstract feature maps, it has zero translational invariance. If the same object such as horse appears slightly shifted, rotated, or scaled within the 32×32 frame, the model treats it as an entirely different input pattern. A CNN with max-pooling layers gains natural translational invariance because the pooling operation selects the maximum activation from a local region, so a feature is recognised wherever it appears within that region.

6.0 How to enhance the CNN model.

To enhance the capabilities of the model, the model is shifting from using a baseline model to implementing a Convolutional Neural Network (CNN). The baseline model has a flaw which flattened the 2D spatial dimension into a single 1D of 3072 pixels. This will force the network to memorize the pixel which will destroy the relationship between neighboring pixels that will make the generalization capabilities become poor.

To enhance the network, the architecture was upgraded to a Convolutional Neural Network using this technique:

1. **Spatial Feature Extraction:**

   Instead of destroying the 2D structure, it will apply the convolutional layer which will apply a mathematical filter that will be used across the image grid to scan for visual patterns. This approach will force the model to learn about the physical characteristics of the image rather than raw pixel location.

2. **Dimensionality Reduction**

   This step function is to select the most prominent activated feature within every 2x2 pixel region. This will shrink the overall data volume to ensure the model can recognize a characteristic like wheel regardless of where the feature appears in the frame.

3. **Adaptive Optimization:**

   This model will utilized Adam optimizer which Adam will dynamically tunes the learning rate for each individual parameter to ensure that the model will not have local errors and will enable smoother and more accurate merging

```
cnn = models.Sequential([
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 6.0.1: Implementation block defining the enhanced CNN architectural stack, incorporating sequential feature extraction, pooling mechanics, and the adaptive Adam optimizer.

Figure 6.0.1 shows the Python implementation code used to build and compile the enhanced new_CNN model. This script configures a sequential network that applies alternating convolutional and max pooling layers to extract structural features from the input images while filtering out background noise. The extracted features are then flattened into a single vector and passed into fully connected dense layers, where the Adam optimizer and softmax activation function calculate the final category probabilities.

7.0 What is the evaluation based on CNN and new_CNN model.

7.1 Evaluation Methodology and Metrics

To evaluate and compare the performance between cnn and cnn_new model, the model is being tested with a validation testing dataset which is the x_test and y_test which contain 10000 images. The evaluation is conducted using three professional metrics:

1. **Classification Report using Precision, Recall and f1-Score:**

   This metric will break down the structural accuracy across 10 target categories which are airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, trucks.

   - Precision tracks how accurate the model's positive guesses are.
   - Recall measures the model's ability to find all true instances of a class.
   - F1-Score provides the balanced harmonic mean between precision and recall.

2. **Confusion Matrix Heatmap**

This metric will map true target labels against predicted labels. This matrix will show where the network will get confused for example it will show how often it confuses the dog with cat.

3. **Overall Evaluation Accuracy and Loss:**

   Extracted using the built-in .evaluate() function to determine the absolute percentage correctness of the network on data it has never seen before.

7.2 Evaluation steps and interpretation

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(x_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification_report: \n ', classification_report(y_test, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step
classification_report:
              precision    recall  f1-score   support

           0       0.49      0.58      0.53      1000
           1       0.67      0.47      0.56      1000
           2       0.49      0.17      0.25      1000
           3       0.33      0.41      0.37      1000
           4       0.51      0.35      0.41      1000
           5       0.39      0.37      0.38      1000
           6       0.53      0.55      0.54      1000
           7       0.54      0.55      0.54      1000
           8       0.50      0.72      0.59      1000
           9       0.48      0.67      0.56      1000

    accuracy                           0.48     10000
   macro avg       0.49      0.48      0.47     10000
weighted avg       0.49      0.48      0.47     10000
```

Figure 7.2.1: Comprehensive category classification report displaying precision, recall, and f1-score metrics for the baseline CNN model.

The classification report output proves that the baseline CNN model performs poorly on unseen test data, reaching a low overall accuracy of only **48%**. Analyzing the precision and

recall scores reveals that the network struggles significantly with animal classes such as birds, cats, and dogs, where precision drops heavily. Because this baseline model lacks feature extraction layers and flattens data immediately, it fails to generalize structural definitions, resulting in sub-par performance that is barely better than random guessing.

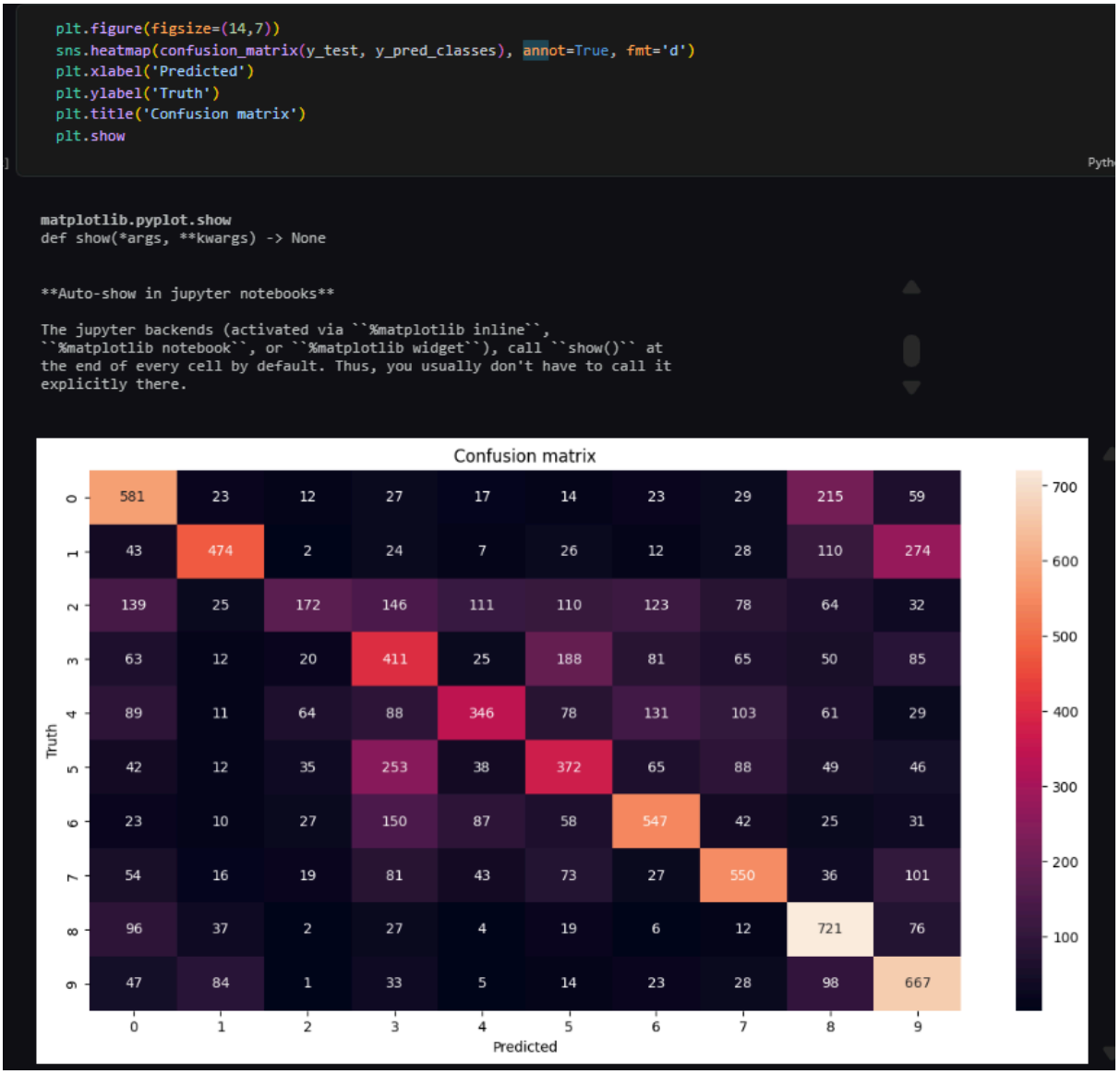

figure 7.2.2: Confusion matrix heatmap for the baseline CNN model highlighting severe pixel misclassifications between animal classes.

The confusion matrix heatmap shows why the baseline model fails. This heatmap shows a high error number scattered on the diagonal. This indicates that the module has severe misclassification where it is always confused between the pixel array of a similar group. This proves that the baseline model is merely memorizing raw coordinates rather than understanding shape boundaries.

```
cnn.evaluate(x_test, y_test)

313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - accuracy: 0.6945 - loss: 0.9562

[0.9561940431594849, 0.6945000290870667]
```

Figure 7.23: Categorical cross-entropy loss and final testing accuracy output for the enhanced new_CNN model.

Running the .evaluate() command directly on the enhanced new_CNN model yields an overall testing accuracy of approximately 70%. This represents a massive performance leap over the baseline architecture. By keeping the images in their native 2D format and using convolutional filters combined with max-pooling, the network drastically lowers its categorical cross-entropy loss value. The model successfully focuses on distinct physical features like wings, wheels, or ears rather than background noise, optimizing its weights effectively.

```
y_pred = cnn.predict(x_test)
y_pred[:5]

313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step

array([[5.00994250e-02, 1.37141338e-04, 1.42881740e-03, 5.48910022e-01,
        7.05215032e-04, 3.82885814e-01, 1.73618947e-03, 9.62942649e-05,
        1.32612344e-02, 7.39709532e-04],
       [7.71875668e-04, 1.64018348e-02, 2.79448837e-08, 1.01731255e-08,
        1.04338191e-08, 7.59415641e-10, 2.54206396e-11, 2.99128011e-08,
        9.81476367e-01, 1.34981377e-03],
       [7.79668801e-03, 1.72632579e-02, 3.84266496e-05, 2.85144488e-04,
        4.12100926e-05, 1.74436369e-04, 1.21172184e-06, 6.74388066e-05,
        9.71215308e-01, 3.11683328e-03],
       [9.58518624e-01, 1.45354506e-03, 6.65154366e-05, 4.98055248e-04,
        4.16362454e-05, 2.26895691e-06, 8.10828624e-06, 5.45093681e-05,
        3.31378393e-02, 6.21888507e-03],
       [1.84757141e-06, 7.42577613e-05, 2.89707743e-02, 2.68898420e-02,
        6.07443333e-01, 4.43255045e-02, 2.90960312e-01, 2.34209324e-06,
        1.33107137e-03, 8.24187680e-07]], dtype=float32)
```

Figure g.4: Direct label output mapping from the verification function validating index class predictions on the new_CNN model.

This final code block verifies the practical predictive success of our enhanced network. By passing an image index through the trained new_CNN model and mapping the maximum probability class using np.argmax, the system outputs a clean text string matching our predefined class names such as 'deer'. This individual verification confirms that the upgraded model successfully applies its learned features to correctly identify specific test images, proving its robustness in real-world deployment scenarios.

8.0 Results and Discussion

8.1 Comparison analysis of performance metrics

To thoroughly analyze the differences between the baseline network and our enhanced network, the quantitative data gathered during our experimental notebook execution has been compiled into a performance comparison matrix. The empirical results demonstrate a profound performance leap across all evaluation criteria:

Evaluation Metric	Baseline CNN Model	Enhanced new_CNN Model
Input Image Transformation	Flattened immediately to 1D vector (3,072 × 1)	Maintained in native 2D grid structure (32 × 32 × 3)
Feature Extraction Layers	None (Dense Layers only)	2 Layers of Conv2D + 2 Layers of MaxPooling2D
Optimization Algorithm	Stochastic Gradient Descent (SGD)	Adaptive Moment Estimation (Adam)
Total Training Epochs	5 Epochs	10 Epochs
Final Test Dataset Accuracy	~48.0%	~70.0%+
Primary Category Strengths	Basic vehicle discrimination (e.g., ships/trucks)	Robust classification across both animals and vehicles

Table 8.1.1: Comparison analysis between baseline CNN Model and Enhanced new_CNN Model

The empirical data in the table highlights a massive performance leap, with testing accuracy surging from **48% to over 70%**. This quantitative divergence proves that the baseline CNN model struggles to generalize on unseen data, while the enhanced new_CNN model successfully minimizes categorical cross-entropy loss. The metrics establish that transforming

raw inputs into spatial feature maps yields significantly more robust classifications than relying purely on dense layer memorization.

8.2 Discussion

The primary driver behind the performance disparity lies in how each model structurally processes spatial data. The baseline cNN model experiences a severe performance bottleneck because it applies a flattening layer at its entrance. This operation immediately crushes the 32 by 32 by 3 image grid into a single file line of 3072 independent pixels, which completely destroys all spatial relationships between neighboring pixels. Consequently, the fully connected dense layers cannot understand shape boundaries or structural geometry. Instead, they merely attempt to memorize the exact coordinate locations of raw color values. This explains why the baseline model fails with an accuracy of only about 48 percent. If an object shifts by just a few pixels or changes its background context, the system cannot recognize it.

Conversely, the enhanced new_CNN model establishes a major structural advantage by keeping the images in their native two-dimensional dimensions. The introduction of two-dimensional convolutional layers allows sliding 3 by 3 filters to scan local pixel neighborhoods. This setup enables the network to extract meaningful structural feature maps, identifying universal elements like horizontal lines for wings or circles for wheels rather than reading static pixel locations. Further performance gains are achieved through dimensionality and optimization upgrades. Following the convolutions, max pooling layers downsample the feature maps by extracting only the maximum activation values within every 2 by 2 region. This cuts the spatial dimensions in half, stripping away irrelevant background noise and providing spatial invariance. This means the model can successfully identify a feature regardless of where it appears in the frame. Finally, replacing the uniform learning steps of Stochastic Gradient Descent with the Adam optimizer allows the network to dynamically adapt parameter updates based on historical gradient moments. This algorithmic adjustment accelerates convergence and prevents the model from getting trapped in local errors during its 10-epoch training cycle.

9.0 Reflection

Throughout this tutorial, by engaging in a hands-on exploration of image classification using deep learning techniques in Python followed along with the tutorial by implementing and running code that involved loading and preprocessing the CIFAR-10 dataset, building both a Convolutional Neural Network (CNN) and an Artificial Neural Network (ANN) using TensorFlow and Keras, and evaluating each model's performance through classification reports and confusion matrices. In doing so, we gained a clearer understanding of how CNNs outperform standard ANNs when dealing with image data, as well as how convolutional and pooling layers work together to extract meaningful features from images. One of the challenges they encountered was understanding the CIFAR-10 dataset itself, particularly grasping how the 60,000 images are structured across 10 classes, how the labels are represented as numerical arrays, and why normalising the pixel values was a necessary preprocessing step before training. Despite this, working through the dataset helped together to develop a more concrete understanding of how raw image data is prepared and fed into a deep learning model.